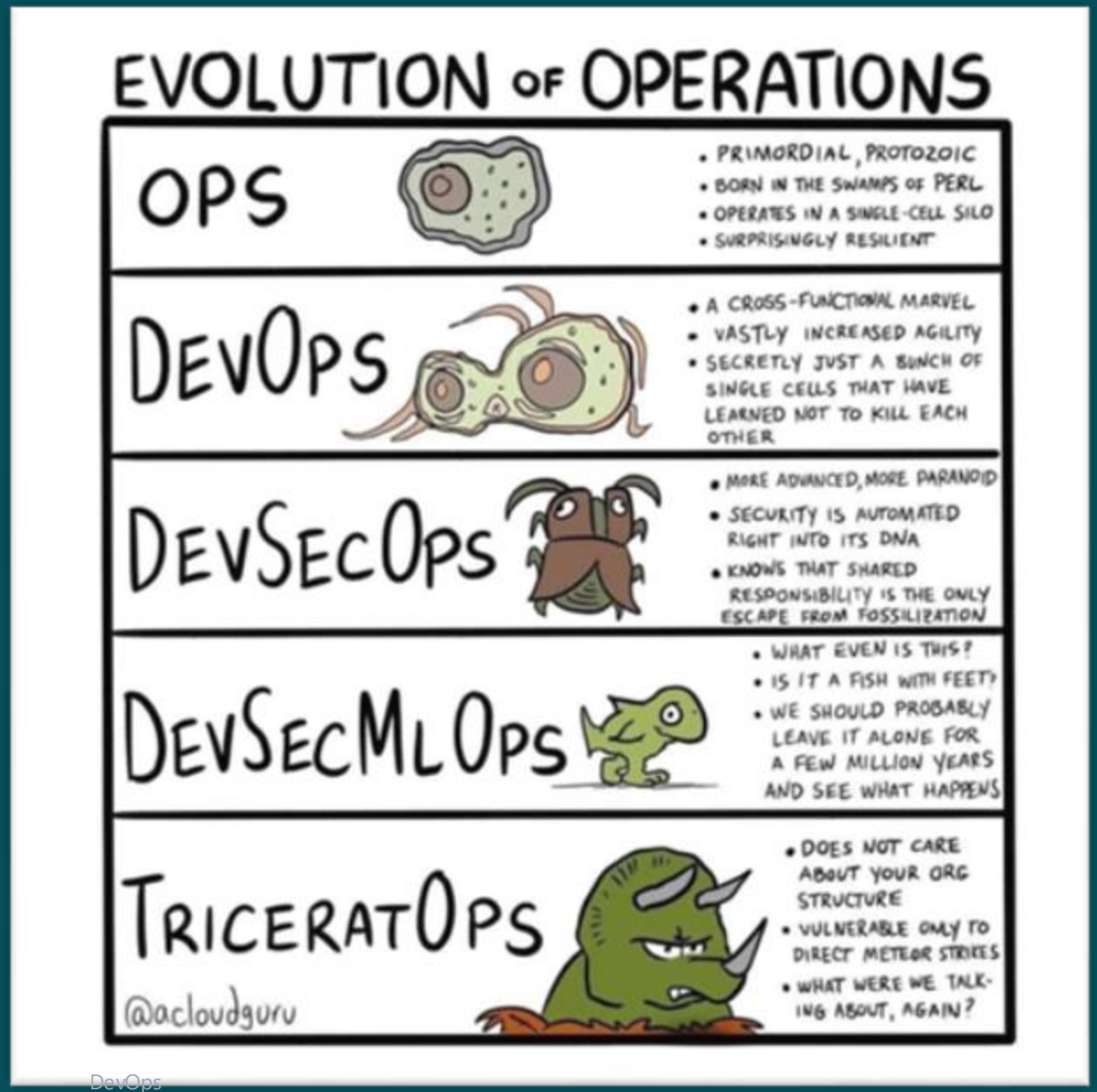
Team DevOps GECIA

Diptarag Saha
Loni Banerjee

Atlas Copco Group

Atlas Copco Group



Agenda

- Introduction to DevOps
- Linux Basics
- SCM – Git/Bitbucket
- Docker
- What is CI/CD
- QnA

Atlas Copco Group

What is DevOps ?

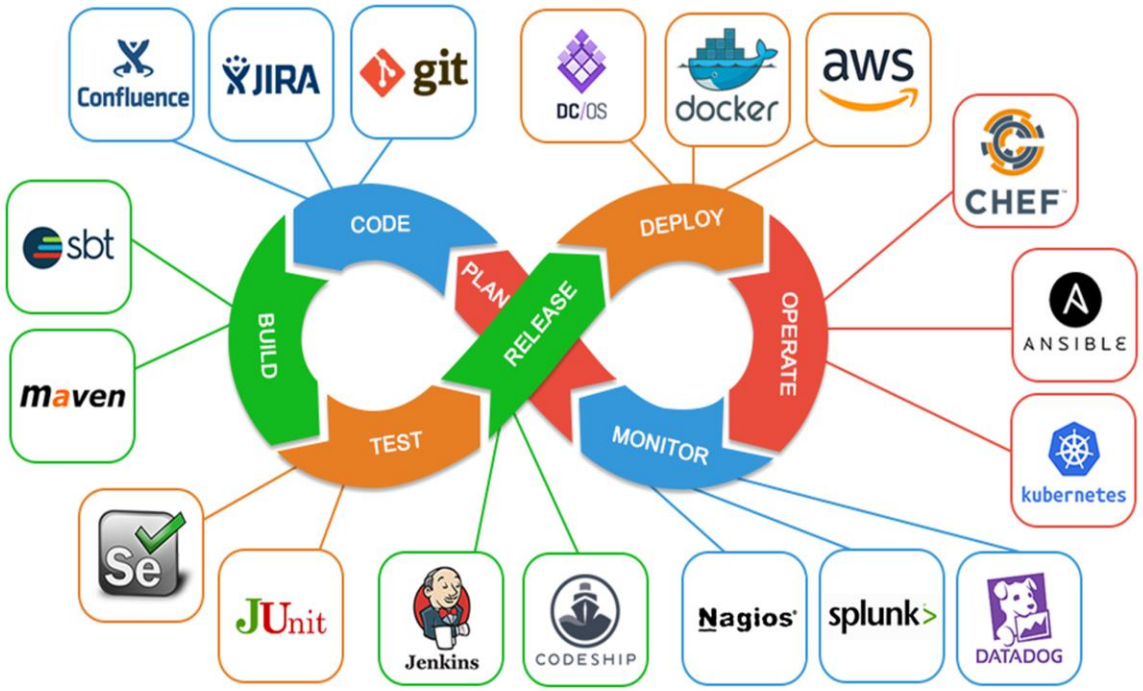
Atlas Copco Group

DevOps

- ***DevOps*** *is a software development and delivery methodology that emphasizes collaboration, automation, continuous integration and continuous delivery to improve the speed, quality and reliability of software lifecycle.*

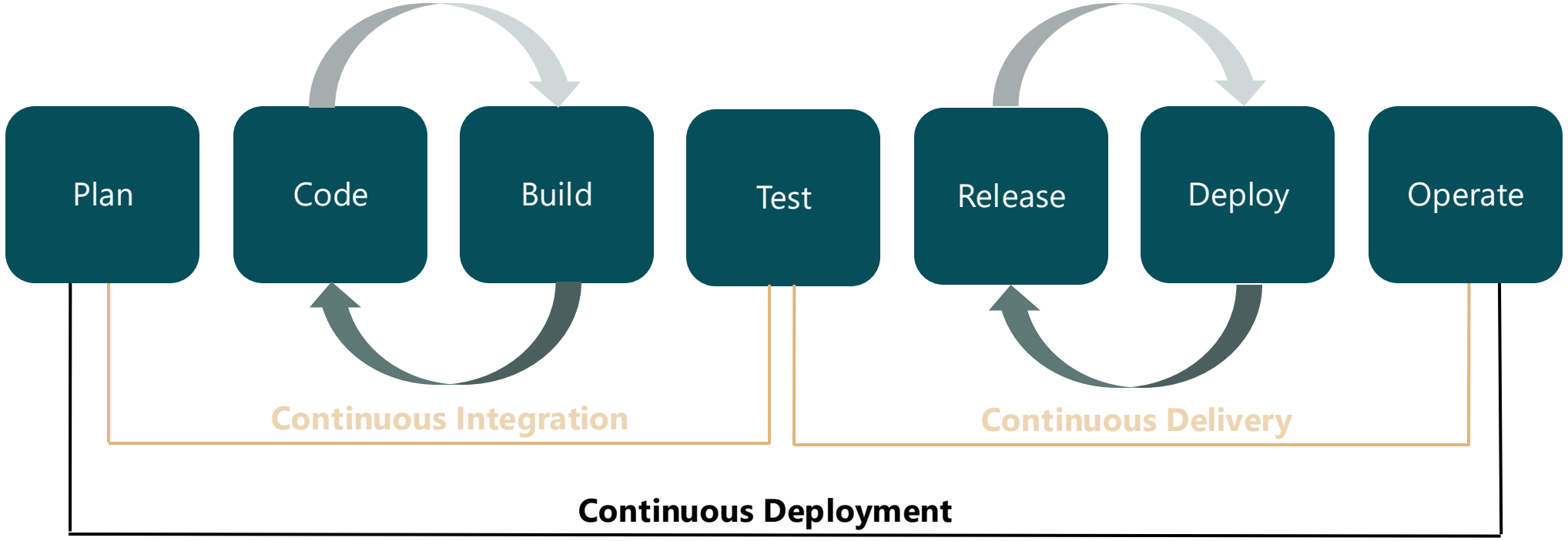
Key Goals:

- Faster software delivery
- Improved Collaboration
- Increased Deployment Frequency
- Reliable releases
- Enhanced quality and security
- Improved monitoring and feedback loop
- Scalability and flexibility



Atlas Copco Group

DevOps Lifecycle Stages



Planning
- Requirement finalization
- Updates & new changes
- Architecture & design
- Task assignment
- Timeline finalization

Code
- Development
- Configuration finalization
- Check-in source code
- Static-code analysis
- Automated review & peer review

Build
- Compile code
- Unit testing
- Code-metrics
- Build container images or package
- Preparation or update in deployment templated
- Create or update monitor dashboards

Test
- Integration test with other component
- Load & stress test
- UI testing
- Penetration testing
- Requirement testing

Release
- Preparing release notes
- Version tagging
- Code freeze
- Feature freeze

Deploy
- Updating the infrastructure i.e staging, production
- Verification on deployment i.e smoke tests

Operate
- Monitor designed dashboard
- Alarm triggers
- Automatic critical events handler
- Monitor error logs

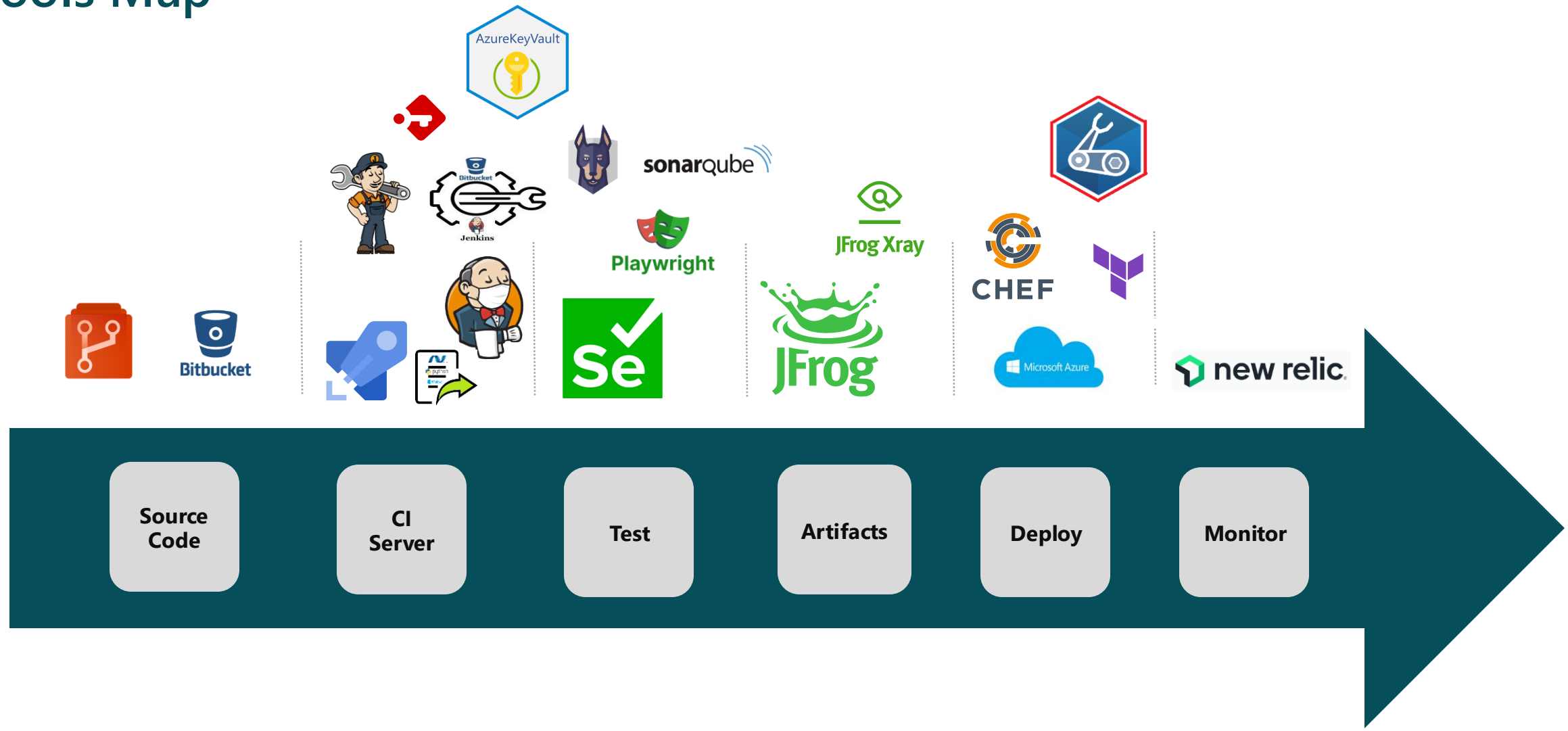
Atlas Copco Group

Competencies & Technologies :

Tools	Used for
Bitbucket	Version Control System
Azure DevOps	Cloud Service for infrastructure
Jenkins	CI/CD Orchestrator. Scripted/Declarative Pipelines.
Artifactory	Artifact storage.
Docker	Build and Run Docker images.
Docker Compose	Deploy docker services
Confluence	Documentation
New Relic	Infrastructure and Application Monitoring
JIRA	Bug Tracking / Project Management
Chef and Ansible	Configuration Management
Python/Bash/Groovy/PowerShell	Automation
Snyk	Security scanning (SCA, SAST, Container vulnerability scanning)
SonarQube	Code Quality

Atlas Copco Group

Tools Map



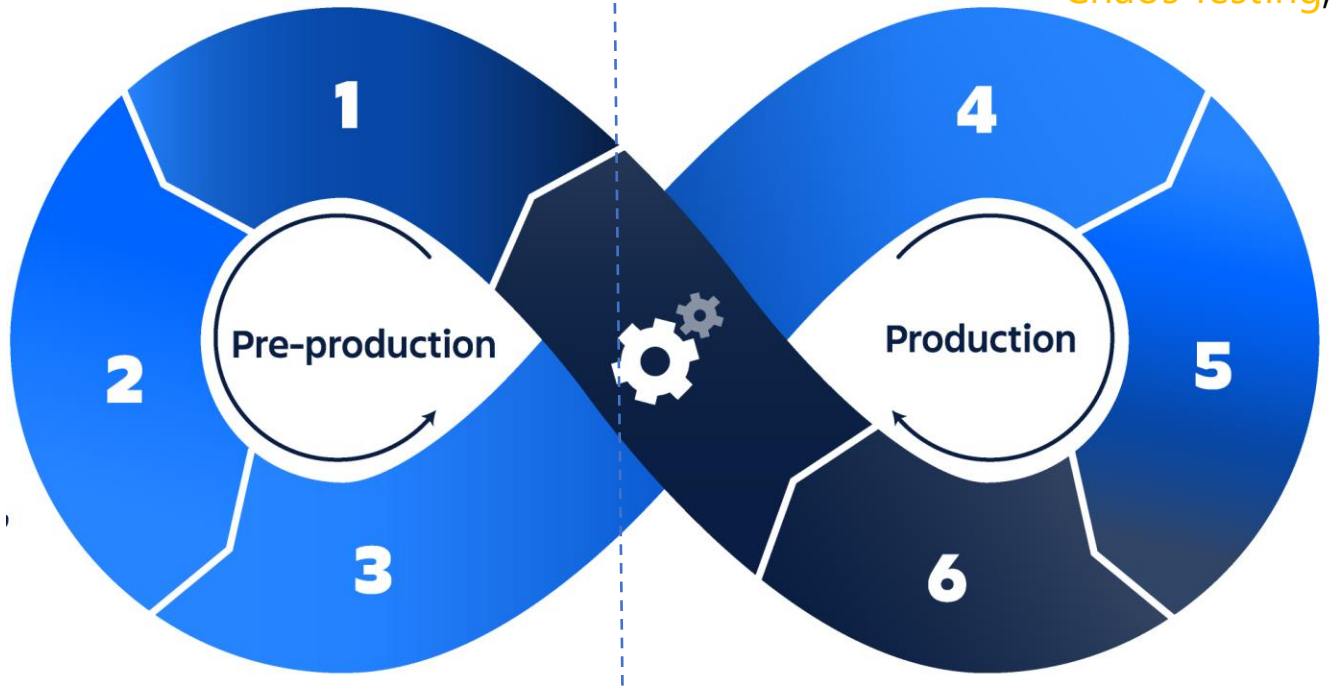
Atlas Copco Group

DevSecOps Activities

Plan
Threat Modelling,
Change Impact analysis

Deploy
Access & Configuration Management,
Chaos Testing, Pen Testing

Build
Pre-commit hooks,
SCA, SAST, Code Review,
Container Security,
Vulnerability scanning



Operate
Log Collection,
WAF

Test
DAST

Monitor
Vulnerability Monitoring,
Access control

Atlas Copco Group

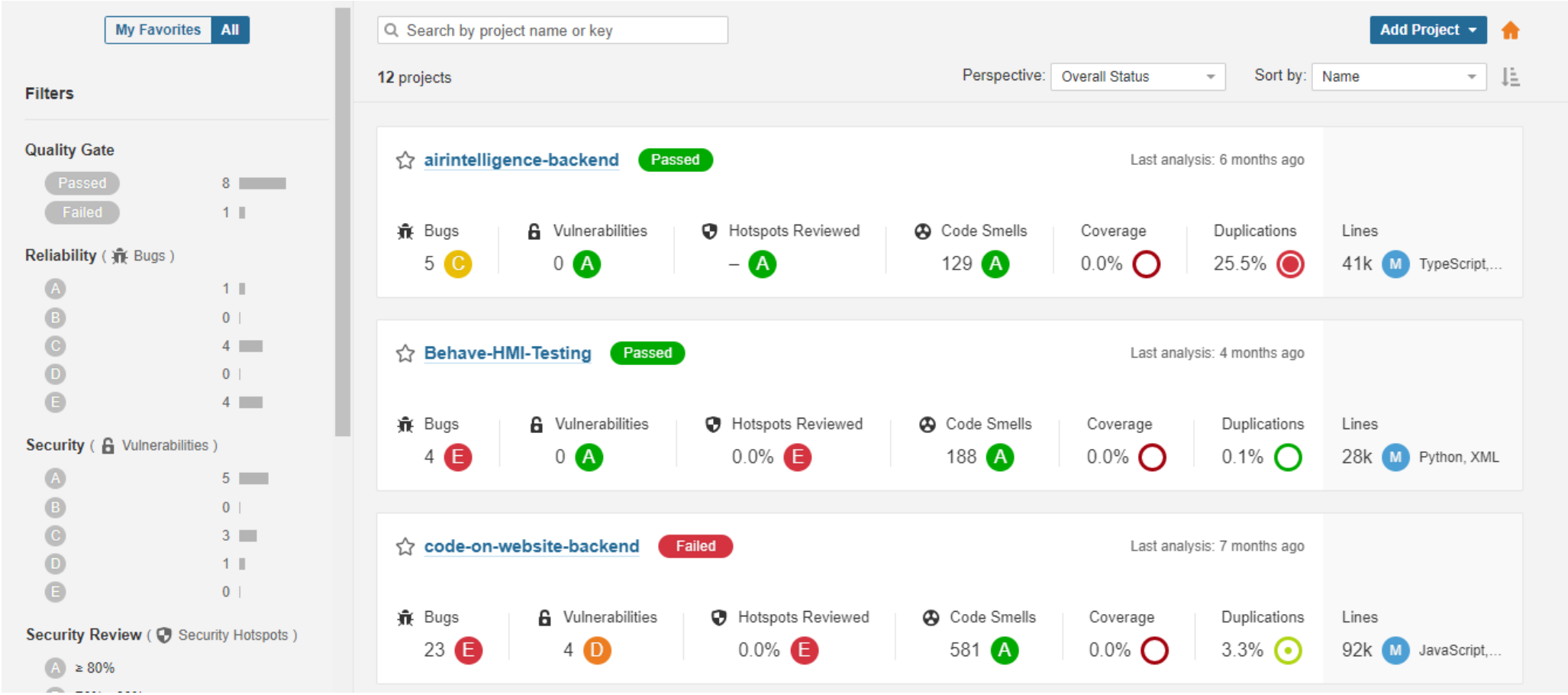
Snyk

https://app.snyk.io/org/airtec

Atlas Copco Group

SonarQube

http://10.76.96.161:9000/projects



Atlas Copco Group

Jenkins Application and Infra

Dashboards

Jenkins - Infra

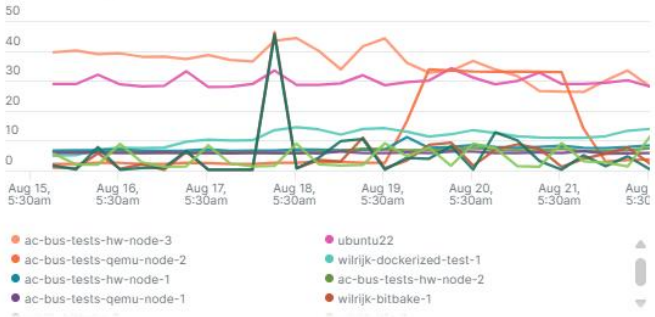
Tags | Teams | ...

Share | Since 7 days ago (GMT+5:30) | Add widget

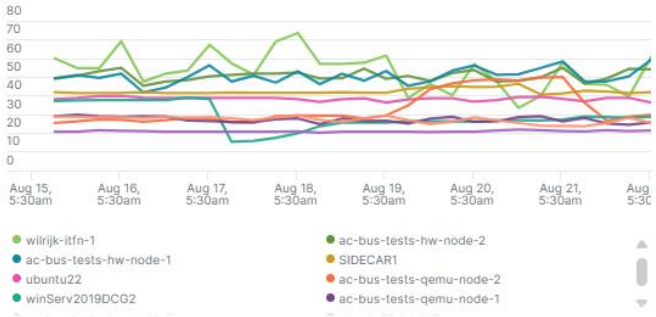
Add variable | Search for any attribute or value.

Static Nodes

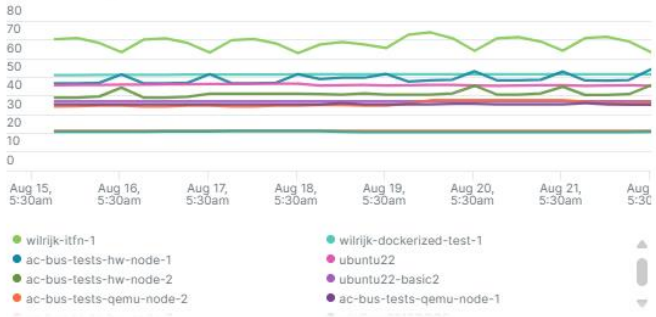
CPU Usage %
Since 1 week ago



Memory Usage %
Since 1 week ago

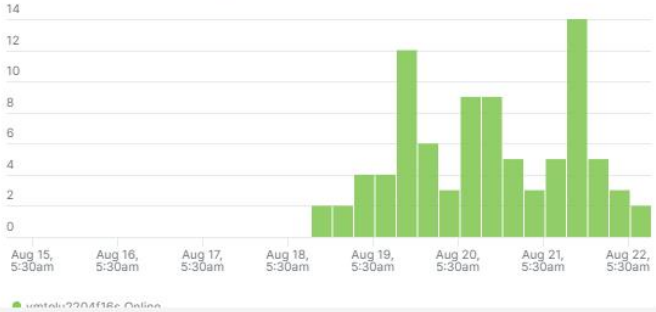


Storage Used %
Since 1 week ago

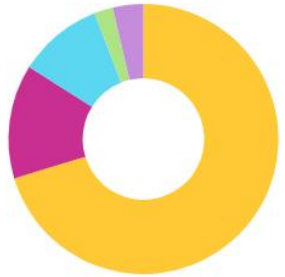


vmtelu2204f8s Online
Aug 15, 12:00pm – Aug 22, 12:00pm

vmtelu2204f16s Online
Aug 15, 12:00pm – Aug 22, 12:00pm



Which Variant has been used most
Aug 15, 12:00pm – Aug 22, 12:00pm



Entity.guids (710)*

Standard_F8s_v2	499	70.28%
Standard_F32s_v2	97	13.66%
Standard_F16s_v2	72	10.14%
Other	26	3.66%
Standard_D32as_v4	16	2.25%

* This pie chart uses uniqueCount, which may result in the facet sum exceeding 100%. Learn more in our documentation.

Dashboard

Atlas Copco Group

Linux Basics

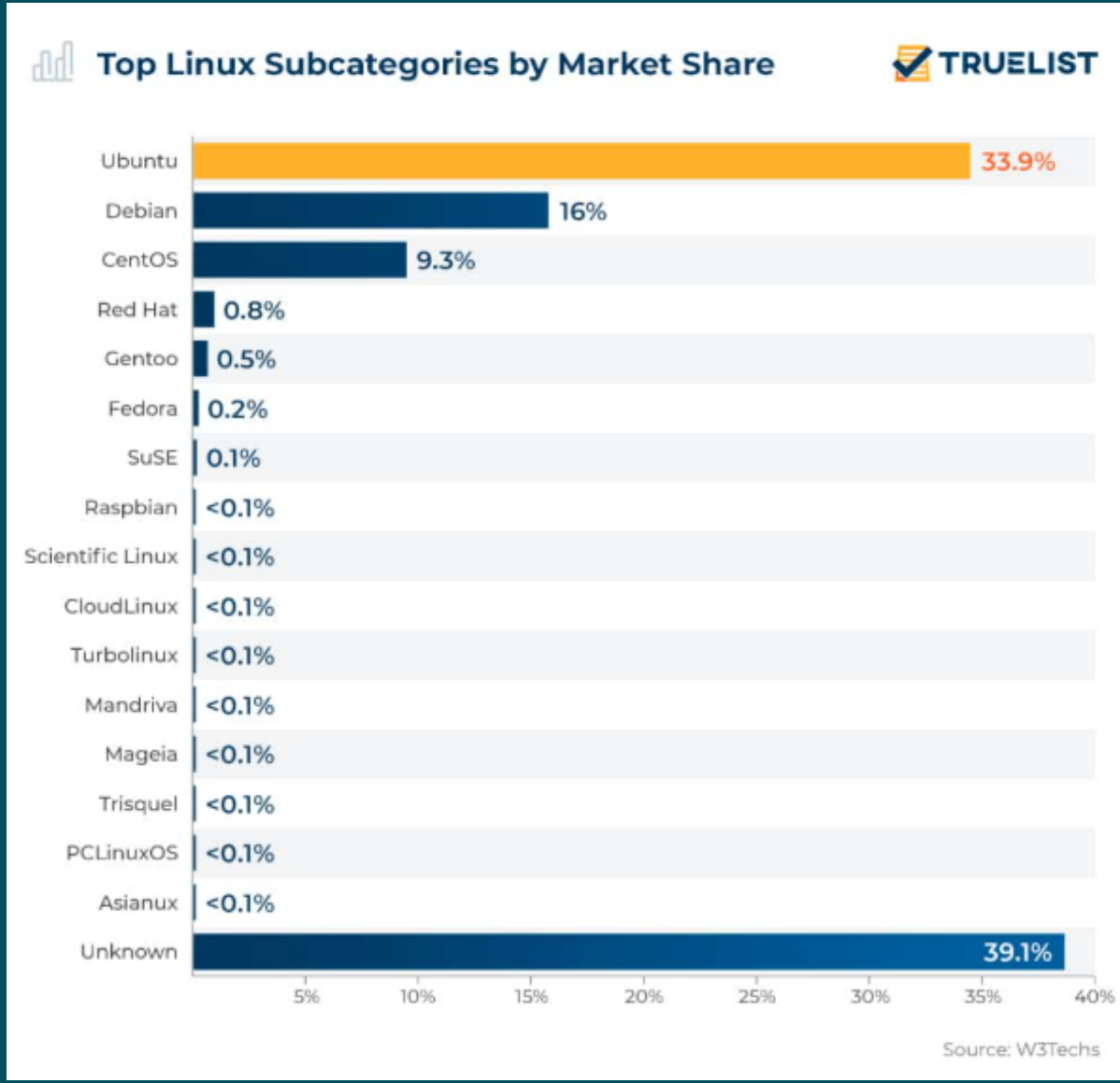
Atlas Copco Group

What is Linux

- - Unix like Operating System
- - Open source and free
- - Kernal + distributions

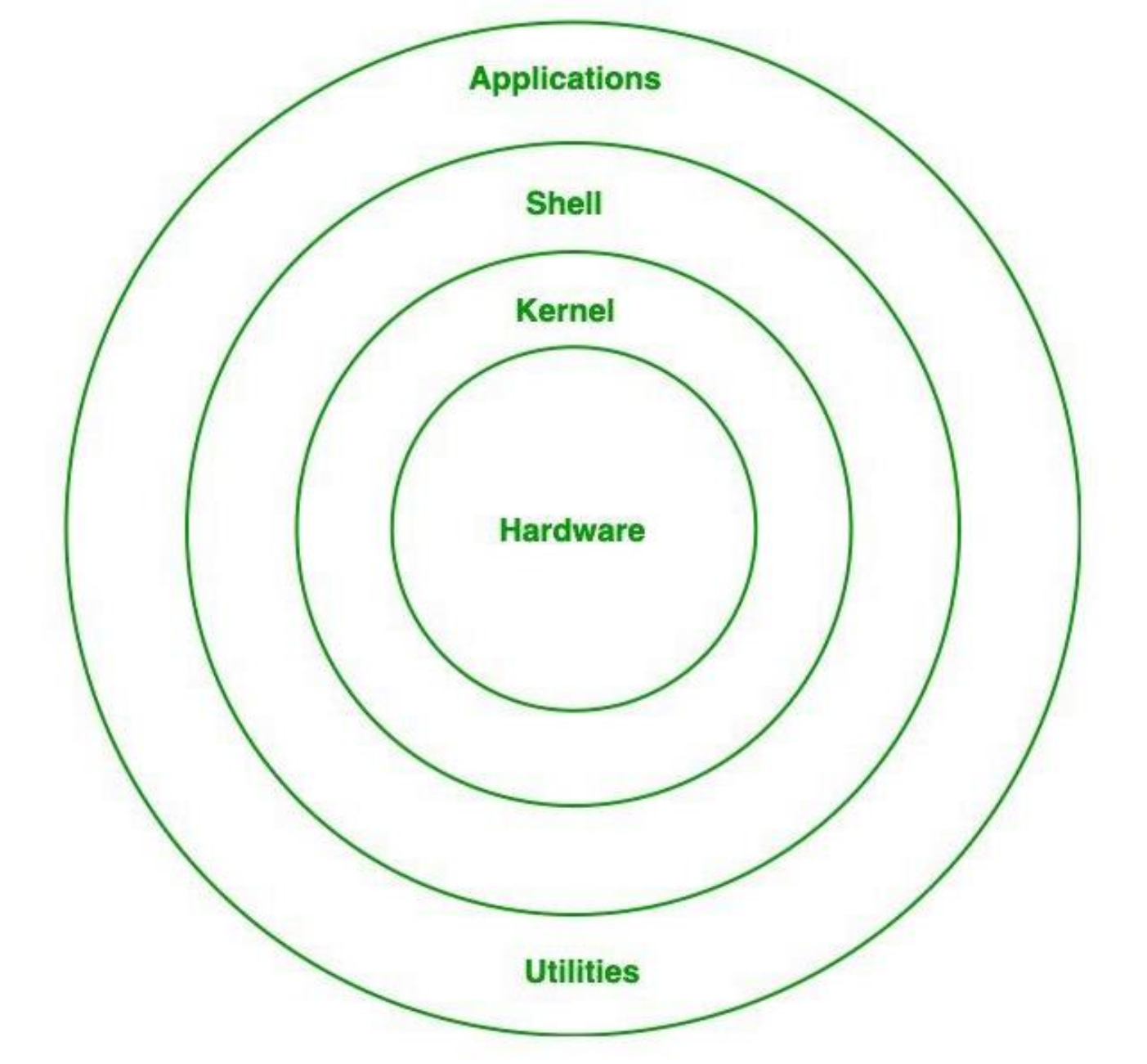
Why learn Linux

- - Stability and Security
- - Open-source freedom
- - Powerful command line
- - industry relevance (cloud, servers)



Atlas Copco Group

Basic Architecture



Atlas Copco Group

File System

```
├── bin/      → Essential binary programs
├── boot/     → Bootloader files (e.g., GRUB)
├── dev/      → Device files (e.g., /dev/sda)
├── etc/      → System configuration files
├── home/     → User home directories
│   └── user/ → e.g., /home/alex
├── lib/      → Shared libraries for binaries
├── media/    → External drives (USB, CD-ROM)
├── opt/      → Optional/3rd-party software
├── proc/     → Kernel and process info (virtual)
├── root/     → Root user's home directory
├── run/      → System runtime data
├── sbin/     → System binaries (for admins)
├── tmp/      → Temporary files
├── usr/      → User programs and data
└── var/      → Variable data (logs, mail, etc.)
```

Atlas Copco Group

Familiarization with Linux Commands

Basic Shell Command:

- ls, cd, pwd, mkdir
- cp, mv, rm, touch
- cat, less, echo
- whoami, chmod, chown
- man
- Text editors – vi, vim, nano
- Find, grep, sed, awk

User Groups and Permissions:

- chmod, chown
- whoami, adduser, passwd
- Read (r), Write (w), Execute (x)

Useful Tools and Editors:

- Text editors: nano, vim, vi
- Networking: ping, netstat, curl
- Monitoring: top, htop, df, du, free

Conditions and loops:

- if – elif –fi conditions
- while – done conditions
- case conditions
- foreach loop

Atlas Copco Group

Linux Demo

Atlas Copco Group

Git Basics

Atlas Copco Group

Introduction

- What are Version Control Systems

1. - Track, manage and control changes to code
2. - Collaborate with others without overwriting work
3. - Restore old versions

- What is Git?

1. - Distributed Version Control System
2. - Fast, reliable and Open source

Atlas Copco Group

Key Git Terminologies

1. **Repository (repo)** – Project folder tracked by Git
2. **Commit** – A saved snapshot
3. **Branch** – A separate line of development
4. **Merge** – Combine changes
5. **Clone, Pull, Push** – Copy and sync code

Atlas Copco Group

Common Git Commands

Command	Description
git init	Start a new Git repo
git clone	Copy a repo
git status	Check current changes
git checkout	Switch branches
git add	Stage files
git commit -m "msg"	Save snapshot
git push	Upload to remote
git pull/git fetch	Get latest from remote
git restore	Undo changes
git merge	Combine changes from one branch to another
git rebase	Combines changes by moving your commits on top of another branch

Atlas Copco Group

Git Demo

Atlas Copco Group

Introduction to Docker

Atlas Copco Group

Why Docker Exists - The Problem Statement

Real-world Analogy: The Moving House Problem

Imagine you're moving to a new house. You pack everything in boxes, but when you reach the new house, your TV doesn't work because the electrical system is different, your furniture doesn't fit because room sizes are different.

Traditional Software Deployment: Applications work on developer's machine but fail in testing/production due to different environments

The Solution: Docker as a "Shipping Container"

Just like shipping containers standardized global trade by providing a consistent way to ship anything anywhere

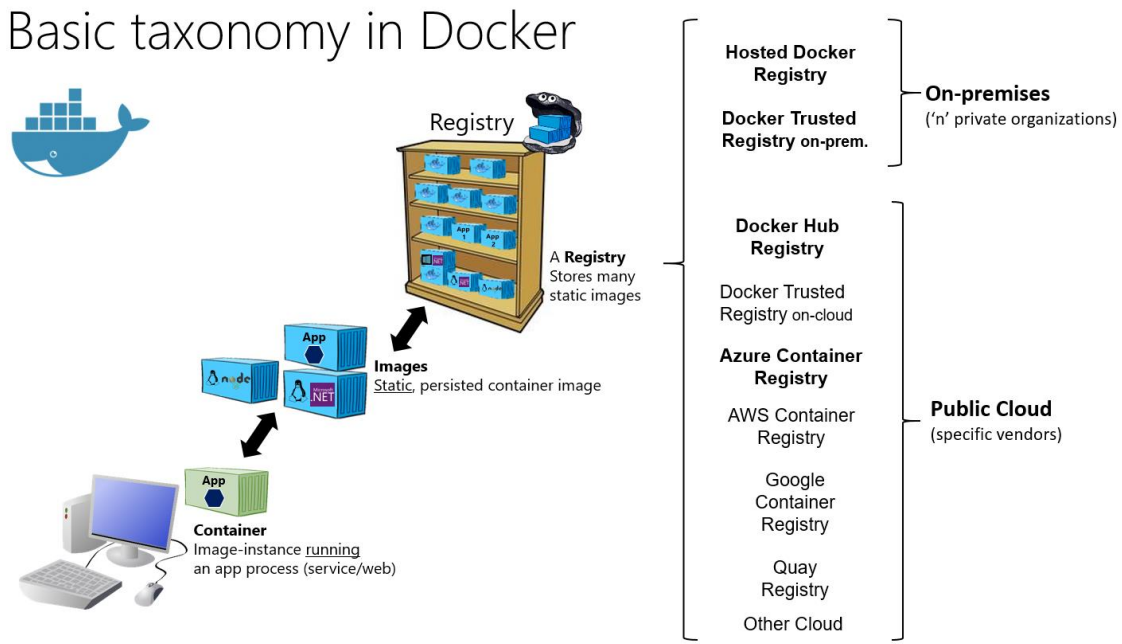
Docker provides a consistent way to package and run applications anywhere

Atlas Copco Group

What is Docker?

Simple Definition: Docker is like a "virtual box" that contains everything your application needs to run - code, libraries, settings, and dependencies.

- **Key Benefits:**
- **Consistency:** "It works on my machine" becomes "It works everywhere"
- **Isolation:** Applications don't interfere with each other
- **Portability:** Run anywhere - laptop, server, cloud
- **Efficiency:** Lighter than virtual machines



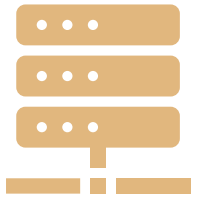
Docker vs Traditional Virtual Machines

Traditional VMs: Like Separate Apartments

Each VM has its own complete operating system

Heavy resource usage

Slow startup



Docker Containers: Like Hotel Rooms

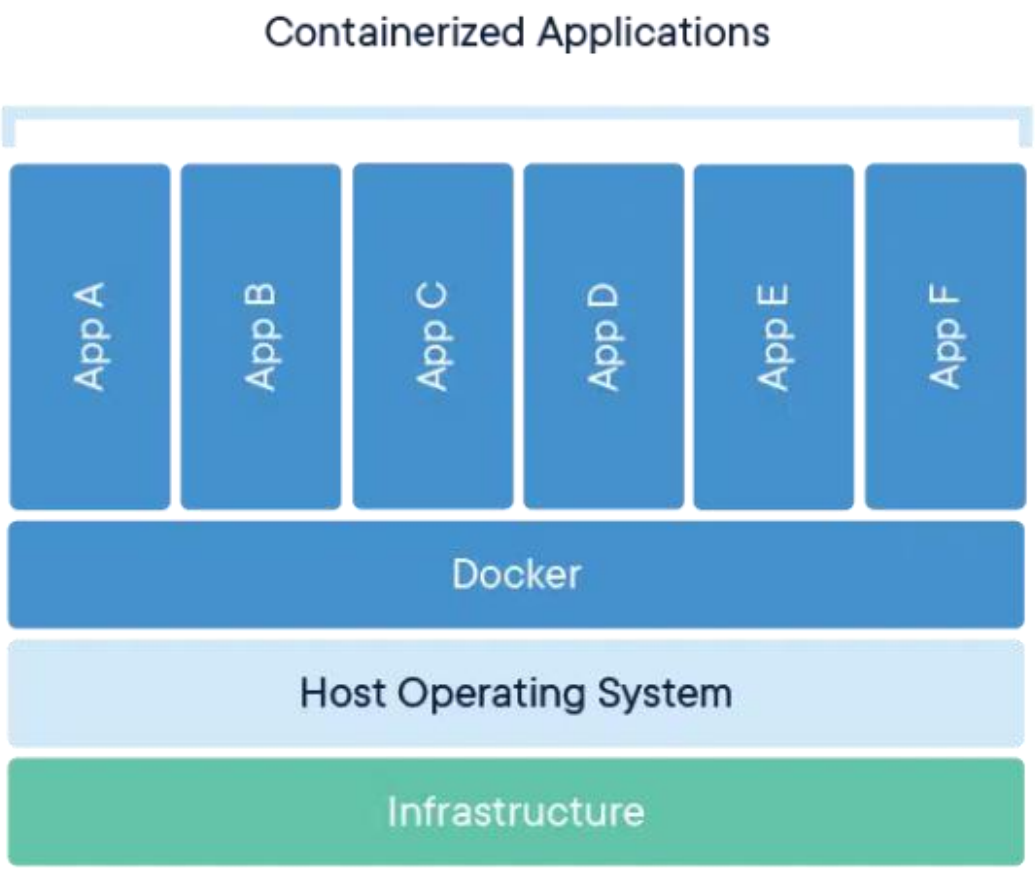
Share the same building (host OS) infrastructure

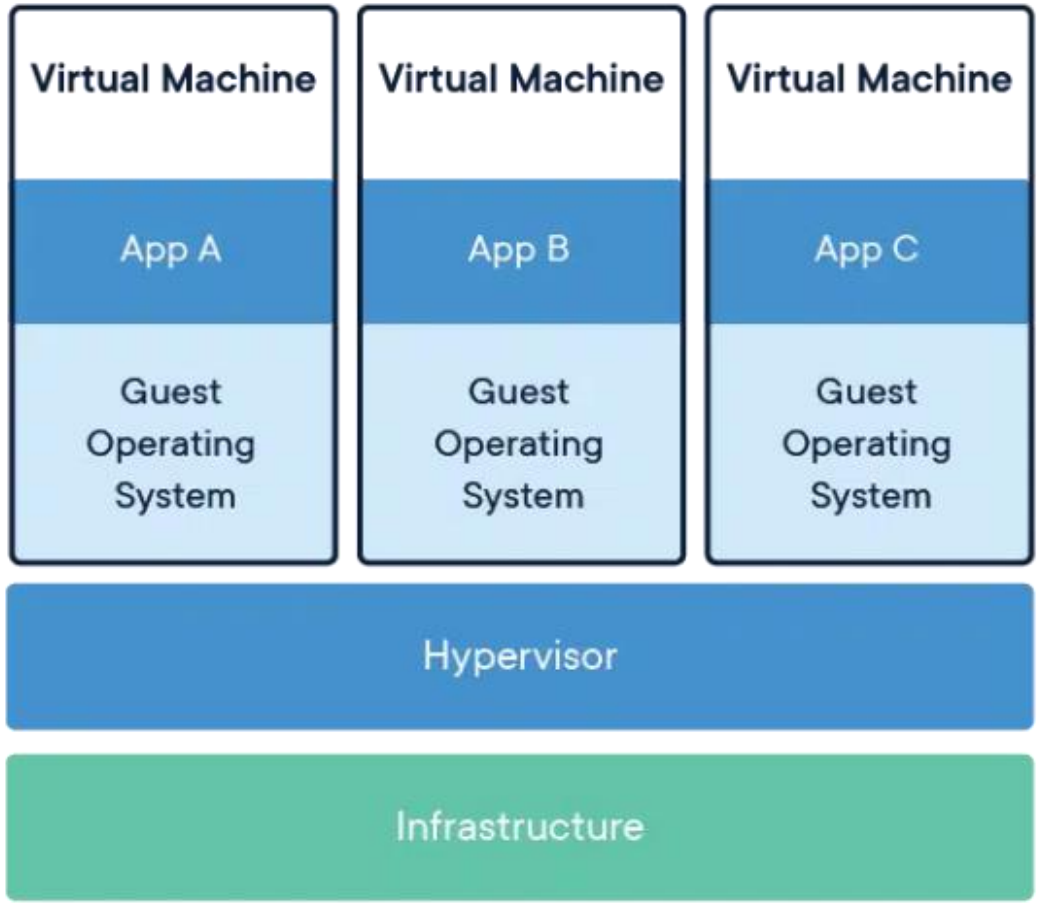
Lightweight and fast

Quick startup (seconds vs minutes)

Atlas Copco Group

Docker vs Traditional Virtual Machines





Atlas Copco Group

Docker Architecture - The Complete Picture

Docker Engine = The Container Factory	Docker Daemon (Background worker) Docker CLI (Your command interface) REST API (Communication bridge)
Docker Images = Blueprints/Templates	Like a recipe for making a dish Read-only templates Layered structure (ingredients added step by step)
Docker Containers = Running Applications	Like a dish made from the recipe Running instance of an image Can be started, stopped, modified
Docker Registry = Recipe Book Library	Docker Hub (public library) Private registries (company cookbook)

Atlas Copco Group

When to Use Docker (and When Not To)

Use Docker When:

Deploying web applications

Microservices architecture

CI/CD pipelines

Development environment standardization

Cloud migrations

Avoid Docker When:

Simple static websites (depends case to case)

Applications requiring direct hardware access

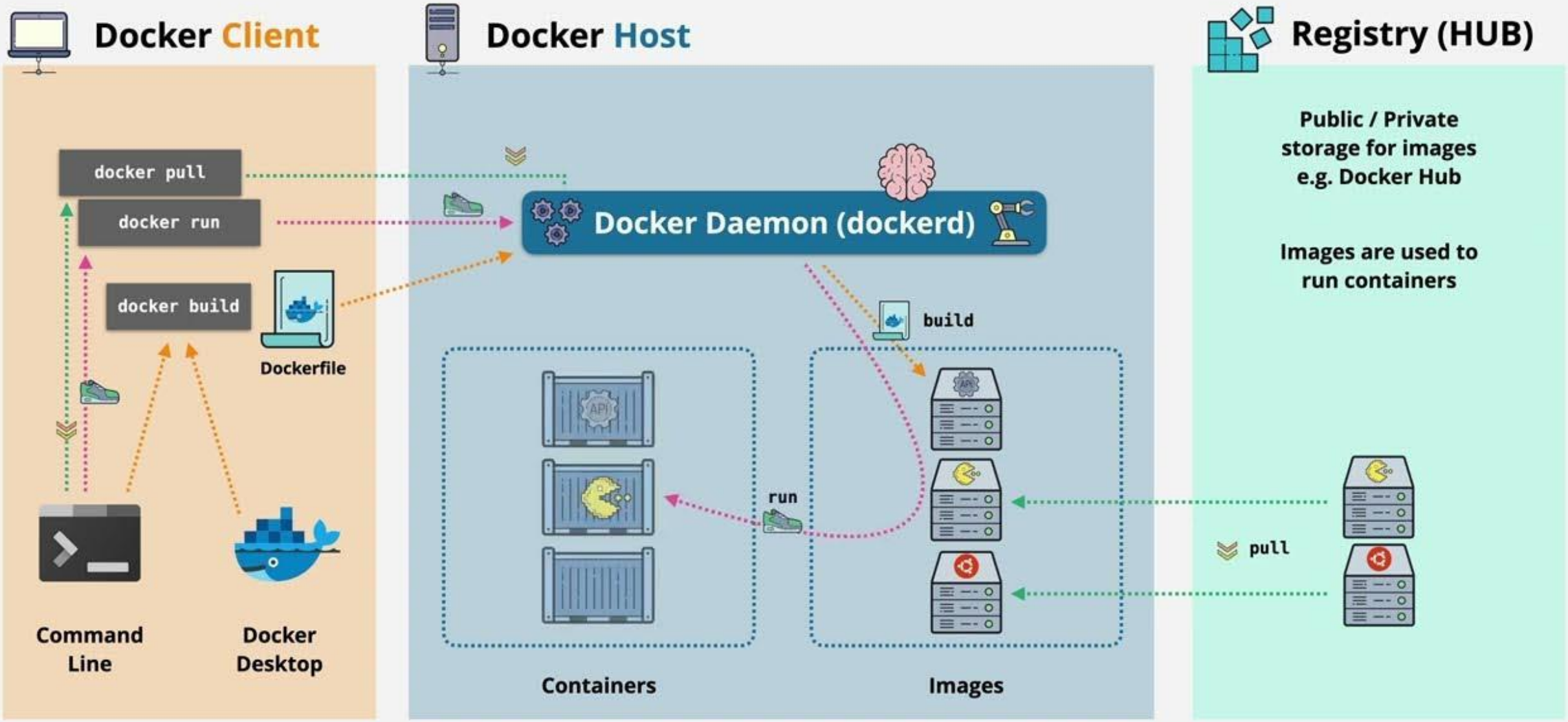
Monolithic applications with complex interdependencies (initially)

Very small teams with simple deployment needs

Atlas Copco Group

Docker Architecture

https://learn.cantrill.io adriancantrill



Live Demo

```
docker version
docker info
docker system df
```


Atlas Copco Group

Essential Docker Commands

Image Management Commands

Analogy: Managing Recipe Books

Image Search and Pull

```
# Search for images (like browsing a cookbook)
docker search nginx
docker search python
# Download images (like getting a recipe)
docker pull nginx
docker pull python:3.9
docker pull ubuntu:20.04
# List local images (your recipe collection)
docker images
docker image ls
```


Atlas Copco Group

Essential Docker Commands

Image Management Commands

Live Exercise 1: Pull different versions of Python images

- docker pull python:3.8
- docker pull python:3.9
- docker pull python:latest
- docker images | grep python

Atlas Copco Group

Essential Docker Commands

Image Information and Management

 # Detailed image information

- docker inspect nginx
- docker image inspect python:3.9

 # Image history (see the recipe steps)

- docker history nginx

 # Remove images (clean up recipes you don't need)

- docker rmi nginx
- docker image rm python:3.8
- docker image prune # Remove unused images

Atlas Copco Group

Essential Docker Commands

Image Management Commands

Live Exercise 2: Inspect and clean up images

- docker pull hello-world
- docker inspect hello-world
- docker history hello-world
- docker rmi hello-world

Atlas Copco Group

Essential Docker Commands

Container Lifecycle Commands

Analogy: Cooking from Recipes

Creating and Running Containers

 # Run container (cook the dish)

docker run hello-world

docker run -it ubuntu:20.04 bash

docker run -d nginx

docker run -d --name my-nginx nginx

 # Run with port mapping (open the restaurant to customers)

docker run -d -p 8080:80 nginx

docker run -d -p 3000:3000 --name my-app node:14

 # Run with environment variables (adjust the recipe)

docker run -d -e MYSQL_ROOT_PASSWORD=password mysql:8.0

Atlas Copco Group

Essential Docker Commands

Container Lifecycle Commands

Live Exercise 3: Run different containers

```
# Interactive Ubuntu container
docker run -it --name my-ubuntu ubuntu:20.04 bash
# Inside container:
apt update && apt install curl -y
# Exit container
# Web server with port mapping
docker run -d -p 8080:80 --name web-server nginx
# Open browser: http://localhost:8080
```


Atlas Copco Group

Essential Docker Commands

Container Status and Management

 # List containers (see what's cooking)

docker ps # *Running containers*

docker ps -a # *All containers*

docker container ls -a # *Container information*

docker inspect my-nginx

docker stats

 # Real-time resource usage

docker logs my-nginx # *Container logs*

docker logs -f my-nginx # *Follow logs*

 # Start/Stop/Restart containers

docker stop my-nginx

docker start my-nginx

docker restart my-nginx

docker pause my-nginx

docker unpause my-nginx

Atlas Copco Group

Essential Docker Commands

Container Status and Management

Live Exercise 4: Container management

```
docker run -d --name test-nginx nginx
docker ps
docker logs test-nginx
docker stop test-nginx
docker ps -a
docker start test-nginx
docker stats test-nginx
```


Atlas Copco Group

Essential Docker Commands

Container Interaction

```
# Execute commands in running containers (modify while cooking)
docker exec -it my-nginx bash
docker exec my-nginx ls /usr/share/nginx/html
# Copy files to/from containers
docker run -d --name my-nginx -p 8080:80 nginx
echo "Hello from Docker!" > file.txt
docker cp file.txt my-nginx:/usr/share/nginx/html/
```

http://localhost:8080/file.txt

Atlas Copco Group

Essential Docker Commands

Container Interaction

Live Exercise 5: Container interaction

```
docker run -d --name web-container nginx
docker exec -it web-container bash
# Inside container: echo "Hello Docker!" > /usr/share/nginx/html/custom.html
docker cp web-container:/usr/share/nginx/html/custom.html ./ cat custom.html
```


Atlas Copco Group

Essential Docker Commands

Cleanup commands

```
# Remove containers (clean up the kitchen)
docker rm my-nginx
docker container rm my-ubuntu
docker container prune # Remove all stopped containers

# Remove everything (complete kitchen cleanup)
docker system prune
docker system prune -a # Include unused images
```


Atlas Copco Group

Essential Docker Commands

Volume and Network Commands

- **Volume Management (Persistent Storage)**

Analogy: Refrigerator Storage

```
# Create volumes (buy a refrigerator)
docker volume create my-data
docker volume ls
docker volume inspect my-data
# Use volumes with containers
docker run -d -v my-data:/data --name data-container ubuntu:20.04
docker run -d -v /host/path:/container/path nginx
# Remove volumes
docker volume rm my-data
docker volume prune
```


Atlas Copco Group

Essential Docker Commands

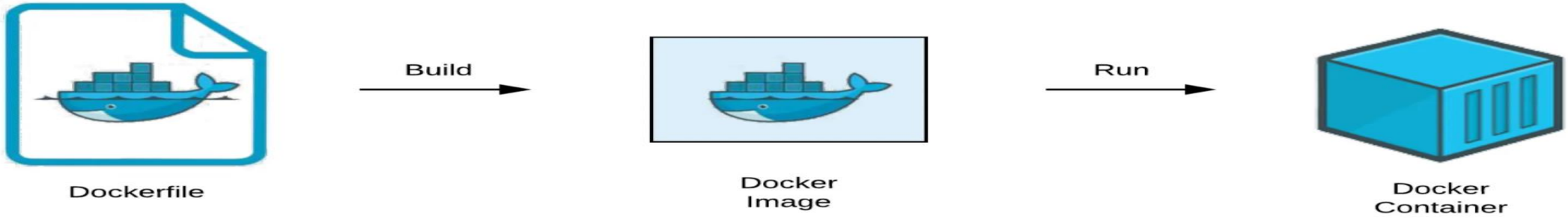
Volume and Network Commands

- Mount Path Examples
- **Analogy: USB Drive or External Folder Access**

```
# Bind mount a local folder
docker run -it --rm -v $(pwd):/app ubuntu:20.04
docker volume ls
docker volume inspect my-data
# Use volumes with containers
docker run -d -v my-data:/data --name data-container ubuntu:20.04
docker run -d -v /host/path:/container/path nginx
# Remove volumes
docker volume rm my-data
docker volume prune
```


Atlas Copco Group

Docker container from docker image




```
# Use an official Python base image
FROM python:3.10-slim
# Set working directory inside the container
WORKDIR /app
# Copy the requirements file
COPY requirements.txt .
# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt
# Copy the rest of the application code
COPY . .
# Expose the port the app runs on
EXPOSE 5000
# Set the default command to run the app
CMD ["python", "app.py"]
```

docker build -t flask-docker-app .

```
mylinux@OVO-aZGCOFvqUO5:~/testflaskimage$ docker images
REPOSITORY
flask-docker-app
```


Atlas Copco Group

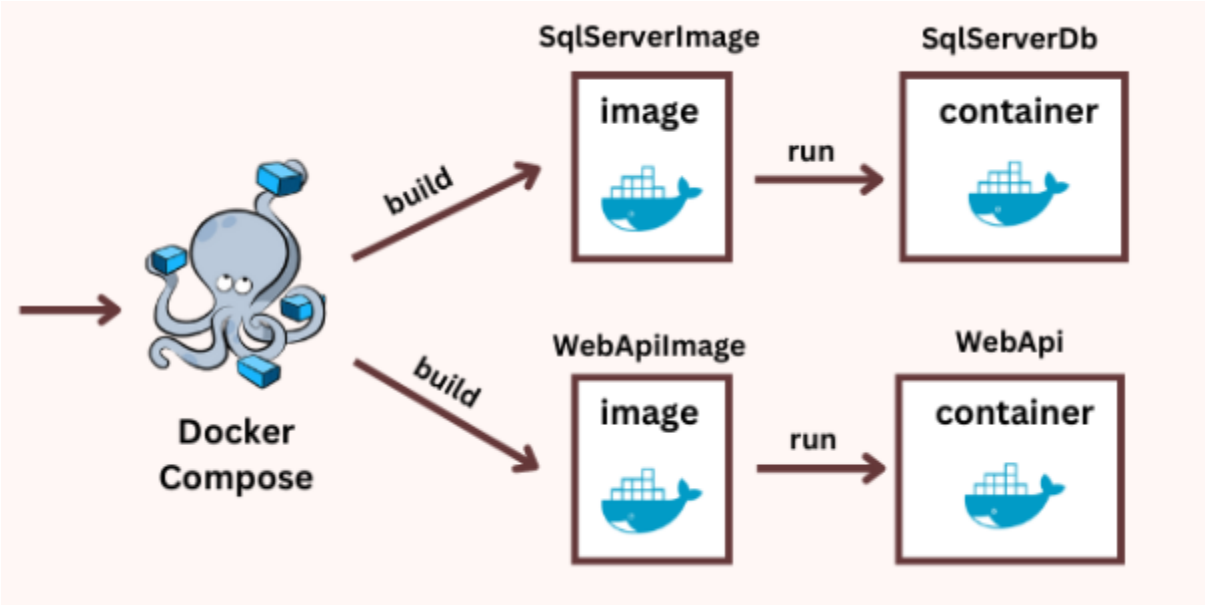
Docker Compose

```yaml
version: '3.8'

services:
  web:
    build: ./app
    ports:
      - "5000:5000"
    depends_on:
      - db
    restart: always

  db:
    image: postgres:15
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: always

volumes:
  pgdata:
```



Atlas Copco Group

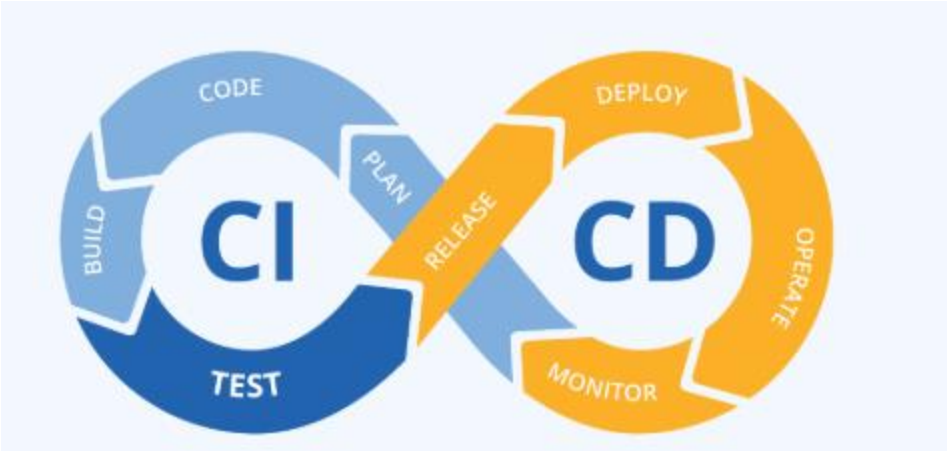
Introduction to CI-CD

Atlas Copco Group

What is CI/CD?

- CI/CD stands for:
  - **CI** = Continuous Integration
  - **CD** = Continuous Delivery / Continuous Deployment

- A DevOps practice that automates the software delivery process.



Why is CI/CD Important?

- Faster and more reliable software releases.
- Quick detection of bugs and issues.
- Better collaboration between Dev and Ops teams.
- Reduced manual work → more automation!

Atlas Copco Group

Overview of CI/CD stages

Continuous Integration (CI)

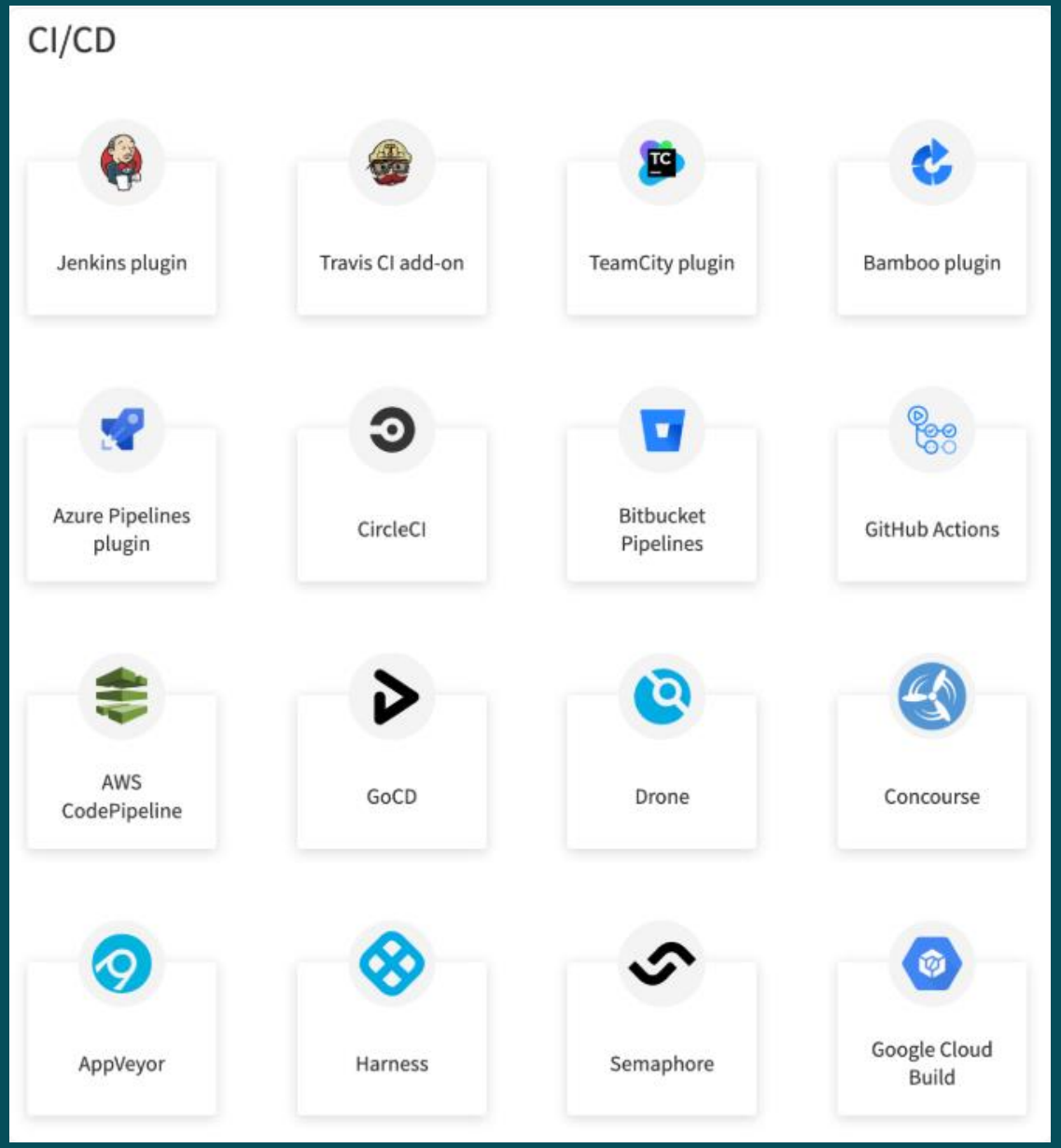
- Developers **frequently push code** to a shared repository.
- Every code push **triggers automated builds/tests**.
- Benefits:
  - Catch bugs early.
  - Avoid integration conflicts.
  - Improves code quality.

Continuous Delivery (CD)

- Builds on CI by **automating deployment** to staging/test environments.
- Code is always in a deployable state.

Atlas Copco Group

Popular CI/CD Tools



- **Jenkins**
- **GitHub Actions**
- **GitLab CI**
- **CircleCI**
- **Azure DevOps**
- **Bitbucket Pipelines**

Atlas Copco Group

Overview of CI/CD stages Contd...

```
pipeline {
    agent any  // Use any available agent to run the pipeline

    stages {
        stage('Checkout Code') {
            steps {
                echo 'Checking out source code...'
                git 'https://github.com/your-org/your-repo.git'
            }
        }

        stage('Build') {
            steps {
                echo 'Building the application...'
                // Example build command (adjust based on your tech stack)
                sh './build.sh'
            }
        }

        stage('Test') {
            steps {
                echo 'Running tests...'
                // Example test command
                sh './run-tests.sh'
            }
        }

        stage('Deploy to Staging') {
            steps {
                echo 'Deploying to staging environment...'
                // Simulate deployment
                sh './deploy.sh staging'
            }
        }

        stage('Deploy to Production') {
            when {
                branch 'main'  // Only deploy to prod from the main branch
            }
            steps {
                echo 'Deploying to production environment...'
                // Simulate deployment
                sh './deploy.sh production'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully!'
        }
        failure {
            echo 'Pipeline failed. Please check the logs.'
        }
    }
}
```


Atlas Copco Group

Overview of CI/CD stages

Continuous Integration (CI)

- Developers **frequently push code** to a shared repository.
- Every code push **triggers automated tests/builds**.
- Benefits:
  - Catch bugs early.
  - Avoid integration conflicts.
  - Improves code quality.

Continuous Delivery (CD)

- Builds on CI by **automating deployment** to staging/test environments.
- Code is always in a deployable state.
- Deployment is **manual** but easy and fast.

Atlas Copco Group

CI-CD Demo

Atlas Copco Group

Atlas Copco
Group

Atlas Copco Group